

MLE5004: Computer systems architecture

I. Data representation: elementary data types, binary representation.

A major stumbling block many beginners encounter when attempting to learn the assembly language is the common use of binary and hexadecimal numbering systems. Although hexadecimal numbers are a little strange, their advantages outweigh their disadvantages by a large margin. Understanding binary and hexadecimal numbering systems is important because their use simplifies the discussion of other topics, including bit operations, signed numeric representation, character codes.

1 Number systems and number representations

Most modern computer systems do not represent numeric values using the decimal (base-10) system. Instead, they typically use a binary or two's complement numbering system.

1.1 Review

Numerical data are expressed in various positional number systems for each radix, or base. A positional number system encodes a vector of n bits in which each bit is weighted according to its position in the vector. The encoded vector is also associated with a radix r , which is an integer greater than or equal to 2. A number system has exactly r digits in which each bit in the radix has a value in the range of 0 to $r-1$, thus the highest digit value is one less than the radix. For example, the binary radix has two digits which range from 0 to 1; the octal radix has eight digits which range from 0 to 7. An n -bit integer A is represented in a positional number system as follows:

$$A = (a_{n-1}a_{n-2}a_{n-3} \dots a_1a_0$$

where $0 \leq a_i \leq r-1$. The high-order and low-order digits are a_{n-1} and a_0 , respectively. The number A can represent positive integer values in the range $[0, r^n - 1]$. So, a positive integer A is written:

$$A = a_{n-1} \cdot r^{n-1} + a_{n-2} \cdot r^{n-2} + \dots + a_1 \cdot r^1 + a_0 \cdot r^0$$

or

$$A = \sum_{i=0}^{n-1} a_i \cdot r^i.$$

Adding 1 to the highest digit in a radix r number system produces a sum of 0 and a carry of 1 to the next higher-order column. Thus, counting in radix r produces the following sequence of numbers:

$$0, 1, 2, \dots, (r-1), 10, 11, 12, \dots, 1(r-1), \dots$$

Table 1: Character sets for different radices

Radix (base)	Character sets for different r
2	0, 1
3	0, 1, 2
4	0, 1, 2, 3
5	0, 1, 2, 3, 4
6	0, 1, 2, 3, 4, 5
7	0, 1, 2, 3, 4, 5, 6
8	0, 1, 2, 3, 4, 5, 6, 7
9	0, 1, 2, 3, 4, 5, 6, 7, 8
10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
11	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A
12	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B
13	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C
14	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D
15	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E
16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

The set of r digits for various radices is given in Table 1. In order to maintain one character per digit, the numbers 10, 11, 12, 13, 14, and 15 are represented by the letters A, B, C, D, E , and F , respectively.

Example Count from decimal 0 to 25 in radix 12. Table 1 indicates that radix 12 contains the following set of twelve digits: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B\}$. The counting sequence in radix 12 is

$$\begin{aligned}
 00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 0A, 0B &= (0 \times 12^1) + (11 \times 12^0) = 11_{10} \\
 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 1A, 1B &= (1 \times 12^1) + (11 \times 12^0) = 23_{10} \\
 20, 21 &= (2 \times 12^1) + (1 \times 12^0) = 25_{10}
 \end{aligned}$$

1.2 The Binary Numbering System

Most modern computer systems operate using binary logic. The computer represents values using two voltage levels (usually $0V$ and $+2.4..5V$). Two such levels can represent exactly two unique values. These could be any two different values, but they typically represent the values 0 and 1. These values, coincidentally, correspond to the two digits in the binary numbering system.

The radix is 2 in the binary number system; therefore, only two digits are used: 0 and 1. The low-value digit is 0 and the high-value digit is $(r - 1) = 1$. The binary number system is the most conventional and easily implemented system for internal use in a digital computer; therefore, most digital computers use the binary number system. There is a disadvantage when converting to and from the externally used decimal system; however, this is compensated for by the ease of implementation and the speed of execution in binary of the four basic operations: addition, subtraction, multiplication, and division. The radix point is implied within the internal structure of the computer; that is, there is no specific storage element assigned to contain the radix point.

The weight assigned to each position of a binary number is as follows:

$$2^{n-1}2^{n-2} \dots 2^32^22^12^0.2^{-1}2^{-2}2^{-3} \dots 2^{-m}$$

where the integer and fraction are separated by the radix point (binary point).

Example The decimal value of the binary number $(1011.101)_2$ is

$$\begin{array}{ccccccc} 2^3 & 2^2 & 2^1 & 2^0 & . & 2^{-1} & 2^{-2} & 2^{-3} \\ 1 & 0 & 1 & 1 & . & 1 & 0 & 1 \end{array} {}_2 = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + \\ (1 \times 2^{-1}) + (0 \times 2^{-2}) + (1 \times 2^{-3}) \\ = 11.625_{10}$$

Digital systems are designed using bistable storage devices that are either reset (logic 0) or set (logic 1). Therefore, the binary number system is ideally suited to represent numbers or states in a digital system, since radix 2 consists of the alphabet 0 and 1. These bistable devices can be concatenated to any length n to store binary data. For example, to store 1 *byte* (8 bits) of data, eight bistable storage devices are required for the value $(01101011)_2 = (107)_{10}$. Counting in binary is illustrated in Table 2, which shows the weight associated with each of the four binary bit positions. Notice the alternating groups of 1s in Table 2 for each of the four columns. A binary number is a group of n bits that can assume 2^n different combinations of the n bits. Therefore,

the **range** for n bits is $[0, 2^{n-1}]$

and the range for four bits is $[0, 2^{4-1}]$; that is 0 to 15, as shown in Table 2.

Table 2: Counting in binary.

Decimal	Binary			
	8	4	2	1
	2^3	2^2	2^1	2^0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

The binary weights for each bit position of an 8-bit integer are shown below

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

1.3 The Hexadecimal Numbering System

Unfortunately, binary numbers are verbose. To represent the value $(202)_{10}$ requires eight binary digits. The decimal version requires only three decimal digits and thus represents numbers much more compactly than in binary. This fact is not lost on the engineers who design binary computer systems. When dealing with large values, binary numbers quickly become unwieldy. Unfortunately, the computer “thinks” in binary, so most of the time it is convenient to use the binary numbering system. Although we can convert between decimal and binary, the conversion is not a trivial task. The hexadecimal (base 16) numbering system solves many of the problems inherent in the binary system. Hexadecimal numbers offer the two features we’re looking for:

- they’re very compact,
- it’s simple to convert them to binary and vice versa.

The *radix* is 16 in the hexadecimal number system; therefore, 16 digits are used, 0 through 9 and A through F, where by convention A, B, C, D, E, and F correspond to decimal 10, 11, 12, 13, 14, and 15, respectively. The low-value digit is 0 and the high-value digit is $(r - 1) = 15(F)$. The weight assigned to each position of a hexadecimal number is as follows:

$$16^{n-1}16^{n-2} \dots 16^316^216^116^0.16^{-1}16^{-2}16^{-3} \dots 16^{-m}$$

where the integer and fraction are separated by the radix point (hexadecimal point).

The decimal value of the hexadecimal number 6A8C.D41616 is obtained by where $r = 16$ and

$$\begin{array}{cccccccc} 16^3 & 16^2 & 16^1 & 16^0 & . & 16^{-1} & 16^{-2} & 16^{-3} & 16^{-4} \\ 6 & A & 8 & C & . & D & 4 & 1 & 6 \end{array} = \begin{array}{l} (6 \times 16^3) + (10 \times 16^2) + (8 \times 16^1) \\ + (12 \times 16^0) + (13 \times 16^{-1}) + (4 \times 16^{-2}) \\ + (1 \times 16^{-3}) + (6 \times 16^{-4}) \\ = 27,276.82846_{10} \end{array}$$

$a_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ and $-m \leq i \leq n - 1$.

When a count of 1 is added to hexadecimal F, the sum is zero and a carry of 1 is added to the next higher-order column on the left. **Note** that when inserting hexadecimal numbers in an assembly language program manually, the first digit must be a number 0 through 9, then the digits A through F, if required, followed by the hexadecimal radix specifier H.

1.4 Conversion between Radices

Binary to decimal Conversion from any radix r to radix 10 is easily accomplished in the following way

$$A = a_{n-1} \cdot r^{n-1} + a_{n-2} \cdot r^{n-2} + a_{n-3} \cdot r^{n-3} + \dots + a_1 \cdot r^1 + a_0 \cdot r^0.$$

The binary number $(1111000)_2$ will be converted to an equivalent decimal number. The weight by position is as follows:

$$\begin{array}{ccccccc} 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 \end{array}$$

therefore

$$\begin{aligned} (1111000)_2 &= (1 \cdot 2^6) + (1 \cdot 2^5) + (1 \cdot 2^4) + (1 \cdot 2^3) + \\ &\quad (0 \cdot 2^2) + (0 \cdot 2^1) + (0 \cdot 2^0) \\ &= (120)_{10} \end{aligned}$$

Hexadecimal to decimal The hexadecimal number $5C2$ will be converted to an equivalent decimal number. The weight by position is as follows:

$$\begin{array}{ccc} 16^2 & 16^1 & 16^0 \\ 5 & C & 2 \end{array}$$

therefore

$$\begin{aligned} (5C2)_{16} &= (5 \cdot 16^2) + (12 \cdot 16^1) + (2 \cdot 16^0) \\ &= (1474)_{16} \end{aligned}$$

Decimal to binary To convert a number in radix 10 to **any other radix** r , repeatedly divide the integer by radix r , then repeatedly multiply the fraction by radix r . The first remainder obtained when dividing the integer is the low-order digit. The first integer obtained when multiplying the fraction is the high-order digit. The decimal number $(186)_{10}$ will be converted to an equivalent binary number.

$186 \div 2 =$	quotient = 93,	remainder = 0	(0 is the low-order digit)
$93 \div 2 =$	quotient = 46,	remainder = 1	
$46 \div 2 =$	quotient = 23,	remainder = 0	
$23 \div 2 =$	quotient = 11,	remainder = 1	
$11 \div 2 =$	quotient = 5,	remainder = 1	
$5 \div 2 =$	quotient = 2,	remainder = 1	
$2 \div 2 =$	quotient = 1,	remainder = 0	
$1 \div 2 =$	quotient = 0,	remainder = 1	

Therefore, $(186)_{10} = (10111010)_2$.

Decimal to hexadecimal The decimal number $(195)_{10}$ will be converted to an equivalent hexadecimal number. The integer is divided by 16 repeatedly.

$195 \div 16 =$	quotient = 12,	remainder = 3	(3 is the low-order digit)
$12 \div 16 =$	quotient = 0,	remainder = 12 (C)	

Therefore, $(195)_{10} = (C3)_{16}$.

Binary to hexadecimal When converting a binary number to hexadecimal, the binary number is partitioned into groups of four bits as the number is scanned from right to left for integers. If the leftmost group of the integer does not contain four bits, then leading zeroes are added to produce a 4-bit hexadecimal digit. The binary number $(11010101000)_2$ will be converted to a hexadecimal number as shown below.

0 1 1 0	1 0 1 0	1 0 0 0
6	A	8

Hexadecimal to binary To convert from hexadecimal to binary, substitute the four binary bits for the hexadecimal digits. The hexadecimal number $(FA97)_{16}$ will convert to a binary number as shown below.

F	A	9	7
1 1 1 1	1 0 1 0	1 0 0 1	0 1 1 1

Convert from radix r_i to any other radix r_j To convert any nondecimal number A_{r_i} in radix r_i to another nondecimal number A_{r_j} in radix r_j , first convert the number A_{r_i} to decimal, then convert the decimal number to radix r_j by using repeated division and/or repeated multiplication. The radix 9 number $(125)_9$ will be converted to an equivalent radix 7 number. First $(125)_9$ is converted to radix 10.

$$\begin{aligned}(125)_9 &= (1 \cdot 9^2) + (2 \cdot 9^1) + (5 \cdot 9^0) \\ &= (104)_{10}\end{aligned}$$

Then, convert $(104)_{10}$ to radix 7.

$104 \div 7 =$ quotient = 14, remainder = 6	(6 is the low-order digit)
$14 \div 7 =$ quotient = 2, remainder = 0	
$2 \div 7 =$ quotient = 0, remainder = 2	

$$(125)_9 = (206)_7 = (2 \cdot 7^2) + (0 \cdot 7^1) + (6 \cdot 7^0) = (104)_{10}.$$

2 Data Organization

In pure mathematics a value's representation may take require an arbitrary number of bits. Computers, on the other hand, generally work with some specific number of bits. Common collections are single bits, groups of 4 bits (called *nibbles*), groups of 8 bits (*bytes*), groups of 16 bits (*words*), groups of 32 bits (*double words* or *dwords*), groups of 64 bits (*quad words* or *qwords*), groups of 128 bits (*long words* or *lwords*), and more. The sizes are not arbitrary. There is a good reason for these particular values. The bit groups commonly used on the Intel 80x86 chips are described below.

2.1 Bits

The smallest unit of data on a binary computer is a single bit. With a single bit, you can represent any two distinct items. Examples include

- 0 or 1,
- *true* or *false*,
- *on* or *off*,
- *male* or *female*,
- *right* or *wrong*.

However, you are not limited to representing binary data types (that is, those objects that have only two distinct values). You could use a single bit to represent the numbers

- 723 and 1,245,
- 6,254 and 5,
- etc.

You could also use a single bit to represent the colors *red* and *blue*. You could even represent two unrelated objects with a single bit. For example, you could represent the color *red* and the number 3,256 with a single bit. You can represent any two different values with a single bit. However, **you can represent only two different values with a single bit**.

Because most items you'll be trying to model require more than two different values, single-bit values aren't the most popular data type you'll use. However, because everything else consists of groups of bits, bits will play an important role in your programs. Of course, there are several data types that require two distinct values, so it would seem that bits are important by themselves. However, you will soon see that individual bits are difficult to manipulate, so we'll often use other data types to represent two-state values.

2.2 Nibbles

A nibble is a collection of 4 bits. It wouldn't be a particularly interesting data structure except for two facts: *binary-coded decimal* (*BCD*) numbers and *hexadecimal* numbers. It takes 4 bits to represent a single *BCD* or *hexadecimal* digit. With a nibble, we can represent up to 16 distinct values because there are 16 unique combinations of a string of 4 bits.

In the case of hexadecimal numbers, the values 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, *A*, *B*, *C*, *D*, *E*, and *F* are represented with 4 bits.

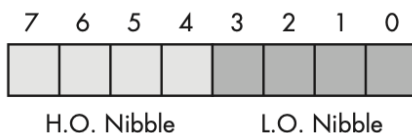
2.3 Bytes

Without question, the most important data structure used by the *80x86* microprocessor is the byte, which consists of 8 bits. Main memory and I/O addresses on the 80x86 are all byte addresses. This means that the smallest item that can be individually accessed by an 80x86 program is an 8-bit value. To access anything smaller requires that we read the byte containing the data and eliminate the unwanted bits. The bits in a byte are normally numbered from 0 to 7, as shown below.



Bit 0 is the *low-order bit* or *least significant bit*, and bit 7 is the *high-order bit* or *most significant bit* of the byte. We'll refer to all other bits by their number.

Note that a byte also contains exactly two nibbles, as shown below. Bits 0...3 compose the *low-order nibble*, and bits 4...7 form the *high-order nibble*. Because a byte contains exactly two nibbles, byte values require two hexadecimal digits.



Because a byte contains 8 bits, it can represent 2^8 (256) different values. Generally, we'll use a byte to represent **numeric values in the range 0...255**, **signed numbers** in the range $-128 \dots +127$ (see later courses), ASCII/IBM character codes, and other special data types requiring no more than 256 different values. Many data types have fewer than 256 items, so 8 bits is usually sufficient.

Because the 80x86 is a byte-addressable machine, it turns out to be more efficient to manipulate a whole byte than an individual bit or nibble. For this reason, most programmers use a whole byte to represent data types that require no more than 256 items, even if fewer than 8 bits would suffice.

For example, the boolean values true and false can be represented by $(00000001)_2$ and $(00000000)_2$, respectively.

Probably the most important use for a byte is holding a character value. Characters typed at the keyboard, displayed on the screen, and printed on the printer all have numeric values. To communicate with the rest of the world, PCs typically use a variant of the ASCII character set. There are 128 defined codes in the ASCII character set.

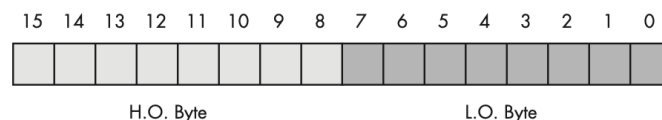
Because bytes are the smallest unit of storage in the 80x86 memory space, bytes also happen to be the smallest variable you can create in an high level language program.

If you want to create an arbitrary byte variable, you should use the byte data type, see listing 1 for an example.

2.4 Words

A word is a group of 16 bits. We'll number the bits in a word from 0 to 15. Like the byte, bit 0 is the *low-order bit*. For words, bit 15 is the *high-order bit*. When referencing the other bits in a word, we'll use their bit position number.

Notice that a word contains exactly 2 bytes. Bits 0...7 form the *low-order byte*, and bits 8...15 form the *high-order byte* (see below).



With 16 bits, you can represent 2^{16} (65,536) different values. These could be the values in the range 0..65,535 or the signed values $-32,768 \dots +32,767$, or any other data type with no more than 65,536 values.

As with bytes, you can also create word variables, see listing 1 for an example.

2.5 Double Words

A double word is a pair of words. Therefore, a double-word quantity is 32 bits long.

Double words (*dwords*) can represent all kinds of different things. A common item you will represent with a double word is a 32-bit integer value (that allows unsigned numbers in the range 0...4,294,967,295 or signed numbers in the range $-2,147,483,648 \dots +2,147,483,647$).

Listing 1: Assembly syntax, declare variables of different type.

```

1  bits 32 ;assembling for the 32 bits architecture
2
3  ; the data segment where the variables are declared
4  segment data use32 class= data
5  ; define variable a as a byte (reserve one byte in memory at address a)
6  ; and assign a value to it
7  a    db    1
8  ; define a word variable
9  b    dw    1
10 ; define a double word variable
11 c    dd    0

```

References

1. A Vancea. Courses, (tank you :))
2. Cavanagh, J. (2013). X86 Assembly Language and C Fundamentals (1st ed.). CRC Press.
<https://doi.org/10.1201/b14582>
3. Randall Hyde. (2020). The Art of Assembly Language (2nd Edition). No Starch Press.
4. Daniel Kusswurm. (2023). Modern X86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX. Apress.